

Math Course Platform - dokumentacja

Projekt: aplikacja webowa do sprzedaży kursów matematycznych online. Jest rejestracja, logowanie, koszyk, lista kursów i panel admina.

Cel projektu

Aplikacja pozwala:

- przeglądać dostępne kursy
- zarejestrować się i zalogować
- dodawać kursy do koszyka
- adminowi zarządzać kursami i userami z osobnego panelu

Technologie użyte w projekcie

- Next.js 16.2.4 (App Router)
- React 19.2.4
- TypeScript
- MongoDB
- Tailwind CSS 4
- bcrypt - do hashowania haseł
- Docker / Docker Compose

Struktura katalogów

```
src/
├── app/
│   ├── (auth)/      # logowanie, rejestracja
│   ├── (main)/      # główne strony
│   ├── api/         # endpointy API
│   ├── admin/page.tsx # panel admina
│   ├── layout.tsx
│   └── page.tsx
├── components/
├── lib/             # mongodb, auth, koszyk
├── scripts/         # seed.js
├── styles/
└── types/
```

Najważniejsze pliki

`src/lib/mongodb.ts` - tu jest połączenie do bazy, funkcja `getDb()` zwraca bazę `mathapp`.

`src/lib/auth.ts` - sprawdza kto jest zalogowany (na podstawie cookie) i czy ma uprawnienia admina.

`scripts/seed.js` - wsadza do bazy konto admina i kilka przykładowych kursów na start.

Endpointy API (każdy w swoim `route.ts`):

- `api/register` - rejestracja
- `api/login` - logowanie
- `api/logout` - wylogowanie
- `api/courses` - lista kursów (publiczna)
- `api/admin/courses` - dodawanie/pobieranie kursów przez admina
- `api/admin/courses/[id]` - usuwanie konkretnego kursu

- `api/admin/users` - lista userów
- `api/admin/users/[id]` - usuwanie usera

Frontend admina to `src/app/admin/page.tsx` razem z komponentem `src/components/AdminPanel.tsx`.

Rejestracja i logowanie

Przy rejestracji trzeba podać name, email i password. Validacja sprawdza czy pola nie są puste, czy hasła się zgadzają i czy hasło ma min. 6 znaków. Email od razu jest zamieniany na małe litery. Login "admin" jest zarezerwowany, więc nie można się tak zarejestrować. Po rejestracji każdy nowy user ma `isAdmin: false` i jest przekierowywany na stronę logowania.

Logowanie działa i na login i na email, więc w formularzu jest jedno pole "login" które przyjmuje obie wartości. Jak się uda, ustawia się ciasteczko `user-session`.

API - lista endpointów

Publiczne

POST /api/register Body: `{ name, email, password }`

POST /api/login Body: `{ login, password }` Ustawia cookie `user-session` jak się uda.

GET /api/logout Czyści cookie, czyli wylogowuje.

GET /api/courses Zwraca wszystkie kursy z kolekcji `courses`.

Admin (wymaga sesji admina)

GET /api/admin/courses - wszystkie kursy

POST /api/admin/courses - nowy kurs, body: `{ title, category, level, price, description, image }`

DELETE /api/admin/courses/[id] - usunięcie kursu

GET /api/admin/users - wszyscy userzy

DELETE /api/admin/users/[id] - usunięcie usera

Każdy z tych endpointów sprawdza najpierw czy user jest zalogowany i czy ma `isAdmin: true`, bo inaczej każdy mógłby usuwać kursy czy userów.

Panel administratora

Strona `/admin` najpierw sprawdza uprawnienia. Jak user nie jest adminem, dostaje info że nie ma dostępu. Jak jest, ładuje się komponent `AdminPanel` gdzie można:

- dodać nowy kurs (formularz z polami title, category, level, price, description, image)
- usunąć istniejący kurs
- zobaczyć listę zarejestrowanych userów
- usunąć usera z listy

Jak działają sesje

Po zalogowaniu w cookie `user-session` zapisuje się id użytkownika. Przy każdym zapytaniu `auth.ts` bierze to id, zamienia na `ObjectId` z mongo i szuka takiego usera w bazie. Funkcja `getAdminUser()` robi to samo plus dodatkowo sprawdza `isAdmin`.

Baza danych

Dwie kolekcje:

users: name, email, passwordHash, isAdmin, createdAt

courses: title, category, level, price, description, image, createdAt

Connection string do mongo:

- w dockerze: `mongodb://mongo:27017/mathapp`
- lokalnie: `mongodb://localhost:27017/mathapp`

Jak uruchomić

Przez Docker (łatwiejsze)

```
cd math-course-main
```

Plik `.env`:

```
MONGODB_URI=mongodb://mongo:27017/mathapp
NODE_ENV=development
```

```
docker-compose up
```

Strona działa na `http://localhost:3000`.

Bez Dockera

```
npm install
```

Plik `.env`:

```
MONGODB_URI=mongodb://localhost:27017/mathapp
NODE_ENV=development
```

Trzeba mieć włączone lokalne mongo. Potem opcjonalnie seed:

```
npm run seed
```

I odpalenie:

```
npm run dev
```

Skrypty z package.json

- `npm run dev` - tryb dev
- `npm run build` - build
- `npm start` - produkcja
- `npm run seed` - wsadza dane startowe do bazy
- `npm run lint` - lint

Konto admina

Po seedzie jest gotowe konto:

- login: admin

- hasło: admin

(zrobione tylko do testów na potrzeby projektu, nie jest to bezpieczne rozwiązanie na produkcję)

Najczęstsze problemy

Port 3000 zajęty:

```
lsof -i :3000  
kill -9 <PID>
```

Mongo / docker nie wstaje: Sprawdź czy Docker Desktop jest odpalony i czy port 27017 nie jest zajęty przez coś innego. Jak nic nie pomaga:

```
docker-compose down && docker-compose up --build
```

Problemy z node_modules:

```
rm -rf node_modules package-lock.json  
npm install
```

Linki

- <https://nextjs.org/docs>
- <https://docs.mongodb.com/>
- <https://tailwindcss.com/docs>
- <https://docs.docker.com/>